Curso
Tendencias Aplicadas de Ingenería de Software

Cache de Datos
Bases de Datos en Memoria

Cache de Datos y Bases de Datos en Memoria

- Durante las últimas dos décadas la evolución del hardware ha incrementado significativamente la capacidad de procesamiento de los servidores.
- Sin embargo, la velocidad de acceso al almacenamiento persistente continúa siendo varios órdenes de magnitud más lenta que la velocidad de acceso a memoria principal.
- Esta diferencia genera uno de los principales cuellos de botella en sistemas de software de gran escala.
- Las aplicaciones modernas —plataformas de comercio electrónico, redes sociales, sistemas financieros, videojuegos online y servicios cloud— deben responder a miles o millones de solicitudes por segundo.

Cache de Datos y Bases de Datos en Memoria

- En estos contextos, consultar permanentemente una base de datos persistente puede provocar latencias elevadas y degradación de la experiencia del usuario.

- Las tecnologías de cache y las bases de datos en memoria surgen como mecanismos arquitectónicos destinados a minimizar estos costos de acceso. Ambas buscan acercar los datos al punto donde serán utilizados, reduciendo la necesidad de operaciones de entrada/salida (I/O) sobre dispositivos persistentes.

El Problema del Acceso a Datos

- Consecuencias arquitectónicas:

  Cuando una aplicación requiere recuperar información desde almacenamiento persistente:

  i. Debe ejecutarse una consulta.
  ii. El motor de base de datos debe localizar los datos.
  iii. Los datos deben ser transferidos desde almacenamiento secundario.
  iv. Deben enviarse por la red.
  v. Finalmente llegan a la aplicación.

- Incluso consultas simples pueden consumir varios milisegundos.

El Problema del Acceso a Datos

- Impacto acumulativo. Supongamos:
  - 1000 usuarios concurrentes.
  - 20 consultas por transacción.

- El sistema puede generar decenas de miles de accesos a la base de datos cada segundo.

- En estas circunstancias la cache deja de ser una optimización y se convierte en una necesidad arquitectónica.

Fundamento teórico de las caches

- La eficacia de las caches se basa en un fenómeno observado en prácticamente todos los sistemas computacionales: el principio de localidad.

- Este principio indica que los accesos futuros a los datos no son aleatorios.

- **Localidad temporal**
  - Si un dato fue utilizado recientemente, existe una alta probabilidad de que vuelva a utilizarse en el corto plazo.

Fundamento teórico de las caches

- Ejemplos:
  - Productos populares de una tienda online.
  - Sesiones activas.
  - Noticias destacadas.
  - Resultados deportivos recientes.
- **Localidad espacial**
  - Si un dato es accedido, los elementos cercanos suelen ser accedidos también.

- Ejemplos:
  - Registros consecutivos.
  - Objetos relacionados.
  - Páginas contiguas.

Fundamento teórico de las caches

- Las caches aprovechan ambos fenómenos para reducir accesos repetitivos a componentes costosos.

- La mayoría de los sistemas exhibe una distribución de accesos tipo Pareto:

- 20% de los datos generan 80% de las consultas.
- En algunos sistemas la concentración es aún mayor.

- Por esta razón, almacenar una pequeña fracción de los datos puede producir una mejora significativa del rendimiento global.

¿Qué es una Cache?

- Una cache es un mecanismo de almacenamiento temporal que conserva copias de datos previamente recuperados con el objetivo de acelerar accesos futuros.
- Desde una perspectiva arquitectónica, una cache representa una táctica orientada al atributo de calidad "performance".
- Funcionamiento general: cuando una aplicación necesita información:
  - Consulta la cache.
  - Si encuentra el dato (cache hit), responde inmediatamente.
  - Si no lo encuentra (cache miss), consulta la fuente original.
  - Almacena el resultado para futuras consultas.

¿Qué es una Cache? (Métricas)

- Métricas importantes: **Hit Ratio**
- Porcentaje de consultas satisfechas por la cache.
- Fórmula:
  - Hit Ratio = Hits / (Hits + Misses)
- Ejemplo:
  - 950 hits
  - 50 misses
  - Hit Ratio = 95%
- **Latencia Promedio**
  - Mide el tiempo medio de respuesta considerando accesos exitosos y fallidos.
- **Throughput**
  - Cantidad de operaciones procesadas por unidad de tiempo.
  - Una cache efectiva aumenta significativamente este indicador.

Bases de Datos en Memoria

- Una base de datos en memoria almacena sus estructuras principales directamente en RAM, eliminando la dependencia del acceso permanente a disco para operaciones normales.
- A diferencia de una cache tradicional, una base de datos en memoria puede actuar como sistema principal de almacenamiento operativo.
- La memoria RAM moderna permite:
  - cientos de gigabytes por servidor.
  - acceso extremadamente rápido.
  - procesamiento masivo de datos.

Bases de Datos en Memoria

- Esto ha permitido que muchas aplicaciones ejecuten operaciones completas en memoria.

- Características
  - Acceso en microsegundos.
  - Estructuras optimizadas para RAM.
  - Índices residentes en memoria.
  - Procesamiento altamente concurrente.

- Ejemplos
  - Redis
  - Hazelcast
  - Etc

- Estas tecnologías permiten construir aplicaciones con tiempos de respuesta extremadamente bajos.

Cache vs Base de Datos en Memoria

- Diferencias Conceptuales
  - Aunque frecuentemente se utilizan como sinónimos, representan conceptos distintos.
- **Cache**
  - Su propósito principal es acelerar el acceso a otra fuente de datos.
- Características:
  - Datos derivados.
  - Información reemplazable.
  - Dependencia de un sistema persistente.
- **Base de Datos en Memoria**
  - Puede constituir la fuente principal de información.
- Características:
  - Administración de estructuras complejas.
  - Replicación.
  - Persistencia opcional.
  - Procesamiento transaccional.
- Toda base de datos en memoria puede utilizarse como cache, pero no toda cache puede actuar como base de datos principal.

Atributos de Calidad Relacionados a Cache

- **Rendimiento**
  - El principal objetivo de las caches es mejorar performance.
- Se reduce:
  - tiempo de respuesta
  - acceso a disco
  - tráfico de red
- **Escalabilidad**
  - Permite soportar más usuarios utilizando la misma infraestructura.
- Beneficios:
  - menos consultas a la base
  - menor saturación de conexiones
  - distribución eficiente de carga

Atributos de Calidad Relacionados a Cache

- **Disponibilidad**
  - Una cache distribuida puede continuar respondiendo solicitudes aun cuando existan degradaciones temporales en componentes persistentes.
- **Elasticidad:** en arquitecturas cloud las caches facilitan:
  - escalado horizontal
  - balanceo de carga
  - respuesta a picos de demanda
- Desde la perspectiva de arquitectura de software, la cache constituye una táctica arquitectónica para mejorar atributos de calidad sin modificar la funcionalidad observable del sistema.

Táctica Arquitectónica: Cache Aside

- Es uno de los patrones más utilizado en aplicaciones empresariales.
- También se denomina Lazy Loading.
- Flujo detallado
  - Usuario solicita dato.
  - Aplicación consulta cache.
  - Si existe:
    retorna dato.
  - Si no existe:
    consulta base de datos.
  - almacena resultado.
  - devuelve respuesta.
- Ventajas
  - Simplicidad
  - No requiere infraestructura compleja.

Táctica Arquitectónica: Cache Aside

- Flexibilidad
  - La aplicación decide qué cachear.
- Eficiencia
  - Solo se almacenan datos efectivamente utilizados.
- Desventajas
  - **Cache Miss Inicial**
    El primer acceso continúa siendo costoso.
  - **Consistencia**
    Puede existir información obsoleta.
  - **Complejidad de invalidación**
    La actualización de datos debe coordinarse cuidadosamente.

Táctica Arquitectónica: Read Through

- La cache se convierte en un intermediario transparente entre la aplicación y la base de datos.
- La aplicación deja de conocer el mecanismo de almacenamiento.
- Beneficios
  - Menor complejidad
  - La lógica de cache se centraliza.
- Uniformidad
  - Todas las aplicaciones utilizan la misma política.
  - Mantenimiento simplificado
  - Cambios futuros afectan solamente a la capa intermedia.

Táctica Arquitectónica: Read Through

- Riesgos
  - Dependencia tecnológica
  - La infraestructura adquiere mayor complejidad.
  - La cache se convierte en componente crítico.

Táctica Arquitectónica: Write Through

- Objetivo
  - Mantener sincronizadas la cache y la base de datos.
- Flujo: cada operación de escritura realiza:
  - Actualización en persistencia.
  - Actualización en cache.
  - Confirmación al cliente.
- Ventajas
  - Alta consistencia
  - Los datos leídos desde cache suelen reflejar la última versión disponible.
  - Menor riesgo de obsolescencia
  - La sincronización es inmediata.

Táctica Arquitectónica: Write Through

- Costos
  - Mayor latencia de escritura
  - La operación no finaliza hasta completar ambas actualizaciones.
- Menor throughput
  - Las escrituras pueden convertirse en cuello de botella.
- Posibles usos:
  - pagos electrónicos
  - sistemas de reservas
  - inventarios críticos
- Representa un claro ejemplo de decisión entre rendimiento y consistencia. A medida que se incrementa la garantía de consistencia, se sacrifica parte de la velocidad y escalabilidad del sistema.

Táctica Arquitectónica: Write Back (Write Behind)

- El patrón Write Back (también denominado Write Behind) busca maximizar el rendimiento de escritura evitando que cada operación tenga que esperar la confirmación de la base de datos persistente.

- A diferencia de Write Through, donde la persistencia ocurre inmediatamente, en Write Back la actualización se registra inicialmente en memoria y posteriormente se sincroniza con el almacenamiento permanente.

Táctica Arquitectónica: Write Back (Write Behind)

- **Funcionamiento**
- Paso 1
  - La aplicación escribe en la cache.
- Paso 2
  - La cache confirma inmediatamente la operación.
- Paso 3
  - Un proceso asíncrono actualiza la base de datos posteriormente.

Táctica Arquitectónica: Write Back (Write Behind)

- Beneficios
  - Muy alta velocidad de escritura
  - La operación finaliza sin esperar acceso a disco.
  - Reducción de I/O
  - Varias escrituras pueden agruparse en una sola operación.
- Mayor throughput
  - Ideal para sistemas con enormes volúmenes transaccionales.
- Riesgos
  - Pérdida de información
  - Si la cache falla antes de persistir los cambios, pueden perderse datos.
- Complejidad operativa
  - Se requieren mecanismos de recuperación.
- Consistencia temporal
  - Los datos pueden estar momentáneamente desactualizados.
- Este patrón representa una implementación práctica de sistemas que privilegian disponibilidad y rendimiento por encima de consistencia inmediata.

Táctica Arquitectónica: Refresh Ahead

- Incluso una cache eficiente puede generar latencias elevadas cuando los datos expiran y deben recuperarse nuevamente desde la fuente persistente.

- Este fenómeno produce picos temporales de respuesta conocidos como:
  - Cache Miss Storms
  - Cold Cache Events

Táctica Arquitectónica: Refresh Ahead

- La cache identifica datos frecuentemente utilizados y los actualiza antes de que expiren.

- Flujo
  - Dato próximo a expirar.
  - Sistema detecta alta frecuencia de acceso.
  - Actualización preventiva.
  - Usuario continúa obteniendo respuestas rápidas.

Táctica Arquitectónica: Refresh Ahead

- Ventajas
  - Menor latencia
    Se reducen los cache misses.
  - Mayor estabilidad
    Los tiempos de respuesta son más predecibles.
  - Mejor experiencia de usuario
    Importante en sistemas interactivos.
- Desventajas
  - Se realizan actualizaciones incluso si luego no son utilizadas.

Replicación de Datos en Memoria

- Una cache centralizada introduce un punto único de falla.

- Si el servidor falla:
  - Se pierde disponibilidad.
  - Se pierden datos temporales.
  - Puede colapsar la base de datos por sobrecarga.
- La información se mantiene en múltiples nodos.
  - Nodo Primario
    - Nodo Réplica A
    - Nodo Réplica B

El Problema de la Invalidación de Cache

- La utilidad de una cache depende de que los datos almacenados sean suficientemente correctos.
- Sin embargo, toda cache genera duplicación de información.
- Cuando la fuente original cambia surge la pregunta:
- ¿Cómo mantener sincronizadas ambas copias?

El Problema de la Invalidación de Cache

- Posibles problemas
  - Datos obsoletos
  - El usuario recibe información incorrecta.
  - Inconsistencia
  - Diferentes usuarios observan estados distintos.
  - Decisiones erróneas
  - La aplicación opera sobre información desactualizada.

El Problema de la Invalidación de Cache

- Estrategias
- **TTL**
  - Expiración automática.

- **Invalidación explícita**
  - Eliminar entradas cuando ocurren cambios.

- **Actualización proactiva**
  - Reemplazar valores inmediatamente.

El Problema de la Invalidación de Cache:TTL

- **TTL** (Time To Live)
- Cada elemento almacenado posee una duración máxima.

- Al finalizar ese período:
  - se elimina
  - o se considera inválido
- Ejemplo
  - Producto 123
  - Valor:USD 250
  - TTL: 300 segundos

El Problema de la Invalidación de Cache: TTL

- Ventajas
  - Implementación simple
  - No requiere coordinación compleja.
- Escalabilidad
  - Funciona bien en sistemas distribuidos.
- Menor acoplamiento
  - No depende de eventos externos.
- Problemas
  - Selección del TTL
  - Demasiado corto: muchos misses
  - Demasiado largo: datos obsoletos

Relación entre Persistencia y Cache

- Arquitectura en Capas: las arquitecturas empresariales modernas suelen separar claramente las responsabilidades.
  - Presentación
  - Servicios
  - Cache
  - Persistencia
  - Base de Datos

Relación entre Persistencia y Cache

- Razón Arquitectónica
  - Cada capa se especializa en una función concreta.

- Persistencia
  - Garantiza durabilidad.
- Cache
  - Optimiza rendimiento.
- Servicios
  - Implementan reglas de negocio.

Responsabilidades de la Capa de Persistencia

- Función principal: conservar información de manera durable y consistente.
- Responsabilidades
  - Durabilidad: los datos sobreviven reinicios y fallos.
  - Integridad: mantener restricciones y reglas.
  - Transacciones: garantizar propiedades ACID.
  - Recuperación: permitir restauración después de incidentes.
  - Auditoría: conservar historial de operaciones.
- Tecnologías habituales
  - PostgreSQL
  - Oracle
  - SQL Server
  - MySQL
  - MongoDB
  - Etc

Responsabilidades de la Capa de Persistencia

- Las bases de datos están diseñadas para:
  - confiabilidad
  - recuperación
  - integridad
- No necesariamente para latencias muy bajas.
- Por esta razón suelen complementarse con caches.

Responsabilidades de la Capa de Cache

- Función principal: reducir el tiempo necesario para acceder a información.
- Responsabilidades
  - Almacenamiento temporal: conservar datos frecuentemente utilizados.
  - Absorción de tráfico: reducir consultas repetitivas.
  - Optimización de recursos: disminuir carga sobre bases de datos.
  - Soporte para escalabilidad: permitir crecimiento sin multiplicar costos.

Responsabilidades de la Capa de Cache

- Qué NO debería hacer una cache
  - Reemplazar completamente la persistencia.
  - Almacenar información crítica sin respaldo.
  - Gestionar reglas de negocio complejas.

- La cache debe considerarse una optimización arquitectónica y no la única fuente de verdad.